

Java EE — Module 4

Servlets Detailed Study Notes

Q1. What is a Servlet? Explain its advantages.

A **Servlet** is a Java class that runs on the server side inside a **web container** (e.g., Apache Tomcat) and handles HTTP requests and responses. It is part of the Java EE specification and resides in the `javax.servlet` and `javax.servlet.http` packages.

A Servlet acts as a **controller** in a web application — it receives a request from the browser, processes it (queries a database, applies business logic), and returns a response (HTML, JSON, redirect, etc.).

How a Servlet fits in the web request flow:



Advantages of Servlets:

- **Platform independent** — Written in Java; runs on any OS with a JVM and a compatible web container.
- **Better performance than CGI** — Servlets are loaded once and stay in memory; each request is handled by a new thread (not a new process like CGI).
- **Robust and secure** — Benefits from Java's strong type system, exception handling, and security manager.
- **Session management** — Built-in support for cookies, URL rewriting, and `HttpSession`.
- **Integration** — Seamlessly integrates with JDBC, JSP, EJB, and all Java EE APIs.
- **Portable** — The same `.war` file deploys on Tomcat, JBoss, GlassFish, etc.
- **Scalable** — Multi-threaded; can serve thousands of concurrent requests efficiently.
- **Full HTTP control** — Can set any HTTP header, status code, cookie, or MIME type.

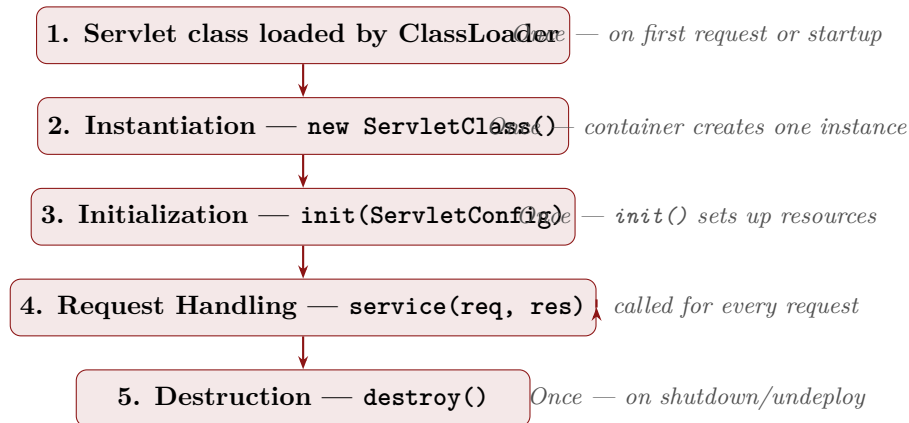
Servlet vs CGI: In CGI, each request spawns a new OS process — expensive and slow. In Servlets, the JVM loads the Servlet class once; each request gets a lightweight thread, making Servlets dramatically faster under load.

Q2. Explain the Servlet lifecycle with diagram.

The **Servlet lifecycle** is the sequence of events from a Servlet's creation to its destruction,

managed entirely by the **web container**. It defines three key methods that every Servlet must implement (inherited from `HttpServlet` / `GenericServlet`).

Lifecycle diagram:



Step-by-step explanation:

- Loading:** When the first request arrives (or at startup if `load-on-startup` is set), the web container loads the Servlet class using the class loader.
- Instantiation:** The container creates exactly **one instance** of the Servlet class using the no-arg constructor. This single instance serves all subsequent requests.
- Initialization — `init()`:** Called **once** after instantiation. The container passes a `ServletConfig` object. Override this method to perform one-time setup (open DB connections, read init parameters, etc.).

```
@Override
public void init(ServletConfig config) throws ServletException {
    super.init(config);
    // one-time setup code here
}
```

- Request Handling — `service()`:** Called **for every incoming request** in a separate thread. `HttpServlet.service()` automatically delegates to `doGet()`, `doPost()`, `doPut()`, etc., based on the HTTP method.

```
@Override
protected void doGet(HttpServletRequest req,
    HttpServletResponse res)
    throws ServletException, IOException {
    res.getWriter().println("Hello from doGet!");
}
```

- Destruction — `destroy()`:** Called **once** when the container shuts down or undeploys the Servlet. Override to release resources (close DB connections, flush buffers, etc.).

```
@Override
public void destroy() {
```

```
    // cleanup code here  
}
```

Thread safety: Because only one Servlet instance exists but many threads call `service()` concurrently, **never store request-specific data in instance variables**. Use local variables inside `doGet()/doPost()` instead.

Q3. Explain GET and POST request handling in Servlets.

HTTP defines several request methods; the two most common in web applications are **GET** and **POST**. `HttpServlet` provides `doGet()` and `doPost()` methods that the container automatically calls based on the HTTP method of the incoming request.

GET Request

- Data is appended to the URL as a **query string**: `?name=Alice&age=20`.
- Visible in the browser's address bar — **not secure** for sensitive data.
- Limited data size (browser/server URL length limits).
- **Idempotent** — repeated GETs produce the same result; safe to bookmark/refresh.
- Used for: searching, filtering, navigation, fetching resources.

```
@WebServlet("/greet")  
public class GreetServlet extends HttpServlet {  
  
    @Override  
    protected void doGet(HttpServletRequest request,  
        HttpServletResponse response)  
        throws ServletException, IOException {  
  
        // Read query parameter from URL: /greet?name=Alice  
        String name = request.getParameter("name");  
  
        response.setContentType("text/html");  
        PrintWriter out = response.getWriter();  
        out.println("<h2>Hello, " + name + "!</h2>");  
    }  
}
```

POST Request

- Data is sent in the **HTTP request body** — not visible in the URL.
- **More secure** for sensitive data (passwords, form data).
- No practical size limit on the request body.
- **Not idempotent** — re-submitting a POST can create duplicate records.
- Used for: form submissions, login, file upload, creating/updating resources.

```

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        // Read form fields from request body
        String user = request.getParameter("username");
        String pass = request.getParameter("password");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        if ("admin".equals(user) && "1234".equals(pass)) {
            out.println("<h2>Login successful!</h2>");
        } else {
            out.println("<h2>Invalid credentials.</h2>");
        }
    }
}

```

Feature	GET	POST
Data location	URL query string	Request body
Visibility	Visible in browser bar	Hidden from URL
Data size	Limited (~2 KB)	Unlimited
Security	Not secure for passwords	More secure
Idempotent	Yes (safe to repeat)	No
Caching	Can be cached	Not cached
Use case	Fetch / search / navigate	Submit form / login / upload

Handling both: To accept both GET and POST with the same logic, have one method call the other:

```

@Override
protected void doGet(HttpServletRequest req,
    HttpServletResponse res)
    throws ServletException, IOException {
    doPost(req, res); // delegate to doPost
}

```

Q4. What is Servlet Deployment Descriptor (web.xml)? Explain its purpose.

The **Deployment Descriptor** is a configuration file named `web.xml`, placed in the `WEB-INF/` directory of a web application. It tells the web container how to deploy and configure the application — which Servlets exist, what URLs they handle, security constraints, filters, listeners, session settings, and more.

In modern Java EE (Servlet 3.0+), annotations like `@WebServlet` can replace most `web.xml` entries, but `web.xml` remains important for enterprise configurations.

Structure of web.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="4.0">

  <!-- Global context parameter (available to all Servlets) -->
  <context-param>
    <param-name>appName</param-name>
    <param-value>MyWebApp</param-value>
  </context-param>

  <!-- Servlet declaration -->
  <servlet>
    <servlet-name>HelloServlet</servlet-name>
    <servlet-class>com.example.HelloServlet</servlet-class>

    <!-- Servlet-specific init parameter -->
    <init-param>
      <param-name>greeting</param-name>
      <param-value>Welcome!</param-value>
    </init-param>

    <!-- Load at startup (0 = load first) -->
    <load-on-startup>1</load-on-startup>
  </servlet>

  <!-- URL mapping for the Servlet -->
  <servlet-mapping>
    <servlet-name>HelloServlet</servlet-name>
    <url-pattern>/hello</url-pattern>
  </servlet-mapping>

  <!-- Default welcome page -->
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>

  <!-- Custom error page -->
  <error-page>
    <error-code>404</error-code>
    <location>/error404.jsp</location>
  </error-page>

  <!-- Session timeout (minutes) -->
  <session-config>
    <session-timeout>30</session-timeout>
  </session-config>

</web-app>
```

Key purposes of web.xml:

- **Servlet registration** — declares which Servlet classes exist and maps them to URL pat-

terns.

- **Initialization parameters** — `<init-param>` for Servlet-specific params; `<context-param>` for app-wide params.
- **Load order** — `<load-on-startup>` controls which Servlets are initialized at container startup.
- **Filter and listener registration** — declares filters and event listeners.
- **Error handling** — maps HTTP error codes or exception types to error pages.
- **Session configuration** — sets session timeout.
- **Welcome files** — default pages shown when a directory URL is accessed.
- **Security constraints** — restricts URL access by role (used with Java EE security).

Annotation vs web.xml: `@WebServlet("/hello")` is simpler for single-Servlet mapping. `web.xml` is preferred when you need centralized configuration, environment-specific overrides, or features not covered by annotations (e.g., complex security constraints).

Q5. What is ServletContext? What are its uses?

ServletContext is an interface in `javax.servlet` that represents the **entire web application**. There is exactly **one ServletContext per web application**, shared by all Servlets, JSPs, and filters within it. It is created when the application starts and destroyed when it shuts down.

Obtained via: `ServletContext ctx = getServletContext();`

Uses of ServletContext:

1. Sharing data across the entire application:

```
// In Servlet A --- store data
ServletContext ctx = getServletContext();
ctx.setAttribute("visitCount", 1000);

// In Servlet B --- retrieve data
Integer count = (Integer) ctx.getAttribute("visitCount");
```

2. Reading context-wide initialization parameters (from web.xml):

```
// web.xml: <context-param><param-name>dbUrl</param-name>
//           <param-value>jdbc:mysql://localhost/mydb</param-value>
//           </context-param>

String dbUrl = getServletContext().getInitParameter("dbUrl");
```

3. Getting the real path of a resource on the server:

```
String realPath = getServletContext().getRealPath("/WEB-INF/config.xml");
```

```
// Returns absolute filesystem path to the file
```

4. Getting a resource as a stream:

```
InputStream is = getServletContext()  
.getResourceAsStream("/WEB-INF/data.json");
```

5. Logging:

```
getServletContext().log("Application started successfully.");
```

6. Getting the RequestDispatcher for forwarding:

```
RequestDispatcher rd = getServletContext()  
.getRequestDispatcher("/dashboard.jsp");  
rd.forward(request, response);
```

Key ServletContext methods:

Method	Purpose
setAttribute(name, value)	Store app-wide data
getAttribute(name)	Retrieve app-wide data
removeAttribute(name)	Remove stored attribute
getInitParameter(name)	Read <context-param> from web.xml
getRealPath(path)	Absolute server path of a resource
getResourceAsStream(path)	Open a resource as InputStream
getRequestDispatcher(path)	Create a dispatcher for forwarding
log(message)	Write to server log
getServerInfo()	Returns web server name/version

Q6. What is ServletConfig? How is it different from ServletContext?

ServletConfig is an interface that provides **configuration information specific to a single Servlet**. The web container creates one **ServletConfig** object per Servlet and passes it to the Servlet's **init()** method. It allows the Servlet to read its own initialization parameters defined in **web.xml**.

Obtained via: `ServletConfig cfg = getServletConfig();`

Using ServletConfig:

```
<!-- web.xml -->  
<servlet>  
<servlet-name>ReportServlet</servlet-name>  
<servlet-class>com.example.ReportServlet</servlet-class>  
<init-param>  
<param-name>reportFormat</param-name>  
<param-value>PDF</param-value>  
</init-param>
```

```

<init-param>
<param-name>maxRows</param-name>
<param-value>500</param-value>
</init-param>
</servlet>

```

```

@Override
public void init(ServletConfig config) throws ServletException {
    super.init(config);
    String format = config.getInitParameter("reportFormat"); // "PDF"
    String maxRows = config.getInitParameter("maxRows");      // "500"
    System.out.println("Format: " + format + ", Max: " + maxRows);
}

```

Key ServletConfig methods:

- `getInitParameter(name)` — returns init param value for this Servlet.
- `getInitParameterNames()` — returns all init param names.
- `getServletName()` — returns the Servlet's registered name.
- `getServletContext()` — returns the shared `ServletContext`.

ServletConfig vs ServletContext:

Feature	ServletConfig	ServletContext
Scope	One specific Servlet only	Entire web application
Instance count	One per Servlet	One per web application
Created by	Container per Servlet	Container at app startup
Parameters	<code><init-param></code> in web.xml	<code><context-param></code> in web.xml
Data sharing	Only within that Servlet	Across all Servlets/JSPs
Accessed via	<code>getServletConfig()</code>	<code>getServletContext()</code>
Lifetime	Servlet's lifetime	Application's lifetime

Memory tip: `ServletConfig` = *private config* for one Servlet. `ServletContext` = *shared blackboard* for the whole application.

Q7. Explain session management techniques in Servlets.

HTTP is a **stateless protocol** — each request is independent and the server has no memory of previous requests. **Session management** is the mechanism used to maintain state (user identity, preferences, cart items) across multiple HTTP requests from the same user.

There are **four main techniques**:

1. Cookies

Small pieces of data stored on the **client's browser** and sent with every request to the same server.

```
// Set a cookie
```



```
Cookie c = new Cookie("userId", "42");
c.setMaxAge(60 * 60); // 1 hour
response.addCookie(c);

// Read cookies
for (Cookie ck : request.getCookies()) {
    if ("userId".equals(ck.getName()))
        System.out.println("User: " + ck.getValue());
}
```

Pros: Simple, persistent across browser restarts (if maxAge set). **Cons:** User can disable cookies; size limited to 4 KB; stored on client (security risk).

2. URL Rewriting

The session ID is appended to every URL as a query parameter. Used when cookies are disabled.

```
// Encode the URL to append session ID automatically
String encodedURL = response.encodeURL("dashboard");
out.println("<a href='" + encodedURL + "'>Dashboard</a>");
// Produces: dashboard;jsessionid=ABC123XYZ
```

Pros: Works without cookies. **Cons:** Session ID visible in URL (security risk); all links must be encoded.

3. Hidden Form Fields

Session data is stored in hidden `<input>` fields inside HTML forms and sent back with each form submission.

```
<!-- Embed user ID in every form -->
<form action="order" method="post">
<input type="hidden" name="userId" value="42" />
<input type="submit" value="Place Order" />
</form>
```

Pros: Works without cookies; no server-side storage. **Cons:** Only works with form submissions; data visible in page source.

4. HttpSession (Recommended)

The container maintains a **server-side session object** identified by a session ID sent to the client via a cookie (JSESSIONID) or URL rewriting. This is the standard, preferred method.

```
// Create / get session
HttpSession session = request.getSession();           // create if none
HttpSession session = request.getSession(false);      // return null if none

// Store data
session.setAttribute("loggedInUser", "Alice");
session.setMaxInactiveInterval(30 * 60); // 30 min timeout

// Retrieve data
String user = (String) session.getAttribute("loggedInUser");
```

```
// Invalidate on logout
session.invalidate();
```

Pros: Secure (data on server); supports any data type; easy API. **Cons:** Consumes server memory per user; requires sticky sessions in clusters.

Technique	Storage	Best for
Cookies	Client browser	Lightweight persistent data
URL Rewriting	URL parameter	Cookie-disabled environments
Hidden Fields	HTML form	Simple form-based flows
HttpSession	Server memory	Secure, full-featured session

Q8. What are cookies? Explain how to create and read cookies in a Servlet.

A **Cookie** is a small piece of text data that a server sends to a client's browser. The browser stores it and **automatically sends it back** with every subsequent request to the same server. Cookies are used for session tracking, user preferences, and remembering login state.

In Java, cookies are represented by the `javax.servlet.http.Cookie` class.

Cookie properties:

- `name` / `value` — the key-value pair stored.
- `maxAge` — lifetime in seconds; `-1` = session cookie (deleted on browser close); `0` = delete immediately.
- `path` — URL path for which the cookie is sent.
- `domain` — domain for which the cookie is valid.
- `secure` — if `true`, sent only over HTTPS.
- `httpOnly` — if `true`, not accessible via JavaScript (XSS protection).

Creating and sending a cookie:

```
@WebServlet("/setCookie")
public class SetCookieServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        // 1. Create the cookie
        Cookie userCookie = new Cookie("username", "Alice");

        // 2. Set properties
        userCookie.setMaxAge(7 * 24 * 60 * 60); // 7 days
        userCookie.setPath("/");                // valid for all paths
        userCookie.setHttpOnly(true);            // not accessible via JS
    }
}
```

```

        // 3. Add to response (browser stores it)
        response.addCookie(userCookie);

        response.getWriter().println("Cookie set successfully!");
    }
}

```

Reading cookies from a request:

```

@WebServlet("/readCookie")
public class ReadCookieServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        Cookie[] cookies = request.getCookies(); // all cookies from browser
        String username = "Guest";                // default

        if (cookies != null) {
            for (Cookie c : cookies) {
                if ("username".equals(c.getName())) {
                    username = c.getValue();
                    break;
                }
            }
        }

        response.setContentType("text/html");
        response.getWriter().println("<h2>Welcome, " + username + "!</h2>");
    }
}

```

Deleting a cookie:

```

// To delete: set maxAge to 0 and add to response
Cookie c = new Cookie("username", "");
c.setMaxAge(0);           // instructs browser to delete it
c.setPath("/");
response.addCookie(c);

```

Security tip: Always set `setHttpOnly(true)` to prevent JavaScript access (XSS attacks), and `setSecure(true)` on production servers to ensure cookies are only sent over HTTPS.

Q9. What is Servlet chaining? Explain with example.

Servlet chaining is a technique where the output (response) of one Servlet becomes the input (request) of the next Servlet in a **chain**. The request passes through multiple Servlets

in sequence, each processing or transforming the data, before the final response is sent to the client.

Servlet chaining is implemented using `RequestDispatcher.include()` or `RequestDispatcher.forward()`.

Two approaches:

1. Using `forward()` — passes full control to next Servlet:

```
// Servlet A --- validates, then forwards to Servlet B
@WebServlet("/servletA")
public class ServletA extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        String data = request.getParameter("input");

        // Process / transform data
        String processed = data.toUpperCase();

        // Pass to next Servlet via request attribute
        request.setAttribute("processedData", processed);

        // Forward to Servlet B
        RequestDispatcher rd =
            request.getRequestDispatcher("/servletB");
        rd.forward(request, response);
        // Note: nothing written to response here ---
        //       Servlet B owns the response completely

    }
}
```

```
// Servlet B --- receives processed data and sends final response
@WebServlet("/servletB")
public class ServletB extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        String data = (String) request.getAttribute("processedData");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h2>Final Result: " + data + "</h2>");

    }
}
```

2. Using `include()` — includes another Servlet's output in current response:

```
@WebServlet("/main")
public class MainServlet extends HttpServlet {
```

```

@Override
protected void doGet(HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();

    out.println("<html><body>");
    out.println("<h1>Main Content</h1>");

    // Include header Servlet's output
    RequestDispatcher rd =
        request.getRequestDispatcher("/headerServlet");
    rd.include(request, response); // output merged here

    out.println("</body></html>");
}
}

```

Method	forward()	include()
Control	Transferred to target Servlet	Stays with calling Servlet
Response	Target writes full response	Output merged into current response
After call	Calling Servlet stops	Calling Servlet continues
Use case	Pipeline / delegation	Composing output from parts

Q10. What are Servlet filters? Explain their purpose.

A **Servlet Filter** is a Java class that intercepts HTTP requests **before they reach a Servlet** and/or HTTP responses **before they reach the client**. Filters implement the `javax.servlet.Filter` interface and are configured in `web.xml` or with the `@WebFilter` annotation.

Filters form a **filter chain** — multiple filters can be applied to the same URL in sequence, each passing control to the next via `FilterChain.doFilter()`.

Filter lifecycle:

1. `init(FilterConfig)` — called once when filter is created.
2. `doFilter(request, response, chain)` — called for every matching request.
3. `destroy()` — called once on shutdown.

Common uses of filters:

- **Authentication / Authorization** — check if user is logged in before allowing access.
- **Logging** — log request details (IP, URL, time) for every request.

- **Input validation** — sanitize or validate incoming parameters.
- **Compression** — GZIP-compress responses for bandwidth saving.
- **Encoding** — set character encoding on all requests.
- **Caching** — add cache headers to responses.
- **CORS headers** — add cross-origin headers to every response.

Example — Logging Filter:

```
@WebFilter("/*")    // applies to ALL URLs
public class LoggingFilter implements Filter {

    @Override
    public void init(FilterConfig config) {
        System.out.println("LoggingFilter initialized.");
    }

    @Override
    public void doFilter(ServletRequest request,
        ServletResponse response,
        FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;

        // --- PRE-processing (before Servlet) ---
        System.out.println("[REQUEST] "
            + req.getMethod() + " " + req.getRequestURI()
            + " from " + req.getRemoteAddr());

        long start = System.currentTimeMillis();

        // Pass request to next filter / Servlet
        chain.doFilter(request, response);

        // --- POST-processing (after Servlet) ---
        long elapsed = System.currentTimeMillis() - start;
        System.out.println("[RESPONSE] completed in " + elapsed + " ms");
    }

    @Override
    public void destroy() {
        System.out.println("LoggingFilter destroyed.");
    }
}
```

Example — Authentication Filter:

```
@WebFilter("/admin/*")    // protects all /admin URLs
public class AuthFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request,
        ServletResponse response,
        FilterChain chain)
```

```

    throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse res = (HttpServletResponse) response;

        HttpSession session = req.getSession(false);
        boolean loggedIn = (session != null
            && session.getAttribute("user") != null);

        if (loggedIn) {
            chain.doFilter(request, response); // allow through
        } else {
            res.sendRedirect(req.getContextPath() + "/login.jsp");
        }
    }

    public void init(FilterConfig c) {}
    public void destroy() {}
}

```

Filter chain order: Multiple filters matching the same URL execute in the order they are declared in `web.xml`. With `@WebFilter`, order is not guaranteed — use `web.xml` when order matters.

Q11. Write the steps to upload and download files using Servlets.

Java Servlets support **file upload** via the `@MultipartConfig` annotation (Servlet 3.0+) and **file download** by setting the correct response headers and streaming the file bytes to the client.

File Upload

Step 1 — HTML form (must use `enctype="multipart/form-data"`):

```

<form action="upload" method="post" enctype="multipart/form-data">
<input type="file" name="myFile" />
<input type="submit" value="Upload" />
</form>

```

Step 2 — Upload Servlet:

```

@WebServlet("/upload")
@MultipartConfig(
    fileSizeThreshold = 1024 * 1024,      // 1 MB before writing to disk
    maxFileSize       = 10 * 1024 * 1024, // max 10 MB per file
    maxRequestSize    = 50 * 1024 * 1024  // max 50 MB per request
)
public class FileUploadServlet extends HttpServlet {

    private static final String UPLOAD_DIR = "uploads";

    @Override
    protected void doPost(HttpServletRequest request,

```

```

    HttpServletResponse response)
    throws ServletException, IOException {

        // Absolute path on server to save file
        String appPath = request.getServletContext().getRealPath("");
        String savePath = appPath + File.separator + UPLOAD_DIR;

        // Create directory if it doesn't exist
        File saveDir = new File(savePath);
        if (!saveDir.exists()) saveDir.mkdirs();

        // Get the uploaded file part
        Part filePart = request.getPart("myFile");
        String fileName = filePart.getSubmittedFileName();

        // Save file to disk
        filePart.write(savePath + File.separator + fileName);

        response.getWriter()
            .println("File uploaded: " + fileName);
    }
}

```

File Download

```

@WebServlet("/download")
public class FileDownloadServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        String fileName = request.getParameter("file"); // e.g. "report.pdf"
        String filePath = getServletContext().getRealPath("/uploads/")
            + fileName;

        File file = new File(filePath);
        if (!file.exists()) {
            response.sendError(HttpServletResponse.SC_NOT_FOUND);
            return;
        }

        // Set headers to trigger browser download dialog
        response.setContentType("application/octet-stream");
        response.setContentLength((int) file.length());
        response.setHeader("Content-Disposition",
            "attachment; filename=\"" + fileName + "\"");

        // Stream file bytes to client
        try (FileInputStream fis = new FileInputStream(file);
            OutputStream out = response.getOutputStream()) {

            byte[] buffer = new byte[4096];
            int bytesRead;
            while ((bytesRead = fis.read(buffer)) != -1) {

```



```

        out.write(buffer, 0, bytesRead);
    }
}
}
}

```

Aspect	Upload	Download
HTTP method	POST	GET
Form enctype	multipart/form-data	N/A
Key annotation	@MultipartConfig	None needed
API used	request.getPart()	response.getOutputStream()
Content-Disposition	N/A	attachment; filename="..."

Security note for upload: Always validate the file extension and MIME type server-side. Never trust the Content-Type header alone. Store uploaded files **outside the webroot** when possible to prevent direct execution.

Q12. Explain request dispatching in Servlets.

Request Dispatching is the mechanism by which one Servlet transfers control of a request to another resource — another Servlet, a JSP, or an HTML file — on the **server side**. It is performed using a `RequestDispatcher` object obtained from either `HttpServletRequest` or `ServletContext`.

Unlike `response.sendRedirect()` (which tells the browser to make a new request), dispatching is entirely **server-side** — the browser URL does not change.

Obtaining a RequestDispatcher:

```

// From request (path relative to current Servlet)
RequestDispatcher rd = request.getRequestDispatcher("/result.jsp");

// From context (path relative to web application root)
RequestDispatcher rd = getServletContext()
    .getRequestDispatcher("/result.jsp");

```

Two dispatching methods:

1. `forward(request, response)`

Transfers **full control** to the target resource. The calling Servlet must not have committed (written) the response.

```

@WebServlet("/process")
public class ProcessServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        // Business logic
    }
}

```

```

        String result = "Processed: " + request.getParameter("data");

        // Share result with JSP via request attribute
        request.setAttribute("result", result);

        // Forward to JSP --- JSP writes the response
        RequestDispatcher rd =
            request.getRequestDispatcher("/result.jsp");
        rd.forward(request, response);

        // Code here is unreachable (or should be avoided)
    }
}

```

```

<!-- result.jsp -->
<html><body>
<h2>${result}</h2>    <!-- reads "result" attribute from request -->
</body></html>

```

2. include(request, response)

Includes the output of the target resource **into the current response**. The calling Servlet retains control and can write more content after the include.

```

@WebServlet("/page")
public class PageServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body>");

        // Include header from another resource
        request.getRequestDispatcher("/header.jsp")
            .include(request, response);

        out.println("<h1>Main Page Content</h1>");

        // Include footer
        request.getRequestDispatcher("/footer.jsp")
            .include(request, response);

        out.println("</body></html>");
    }
}

```

forward() vs **sendRedirect()** vs **include()**:

Feature	<code>forward()</code>	<code>include()</code>	<code>sendRedirect()</code>
Side	Server-side	Server-side	Client-side
Browser URL	Unchanged	Unchanged	Changes to new URL
Request object	Same request	Same request	New request
Response written	By target	Merged into current	By new URL
Use case	MVC forwarding	Composite pages	Post/Redirect/Get

MVC pattern: Request dispatching is the backbone of MVC in Java EE. The Servlet (Controller) handles logic and uses `forward()` to pass control to a JSP (View), keeping business logic and presentation cleanly separated.